Lecture 03: Arrays & Algorithm Analysis

TKU211231 — Student Notes

Syukron Abu Ishaq Alfarozi
Department of Electrical and Information Engineering
Universitas Gadjah Mada

February 26, 2026

Contents

1 Arrays Revisited: Contiguous Memory **3**
1.1 Static Arrays . . . 3
1.2 Dynamic Arrays . . . 4

2 Algorithm Analysis: Big-O Notation **6**
2.1 Definition . . . 6
2.2 Common Complexity Classes . . . 6
2.3 Array Operation Complexities . . . 6
2.4 Quick Analysis Practice . . . 7

3 Exact Operation Counting **8**
3.1 Worked Example: Double-Nested Loop . . . 9

4 Amortized Analysis: The Doubling Strategy **10**

5 Worked Example: The 3-Sum Problem **11**
5.1 Naive Approach — $O(n^3)$ . . . 11
5.2 Optimised Approach — $O(n^2)$: Sort + Two Pointers . . . 12

6 Recurrence Relations and Merge Sort **13**
6.1 Merge Sort Recurrence . . . 13
6.2 Solving the Recurrence: Recursion Tree Method . . . 13
6.3 Recurrence Patterns Summary . . . 14

7 Implementing a Resizable Array (Procedural Style) **14**
7.1 Interface Design . . . 15
7.2 Full Implementation . . . 15
7.3 Critical Bug: Insertion Direction . . . 18

8 Summary **19**
8.1 Complexity Summary Table . . . 19
8.2 Common Pitfalls Checklist . . . 19
8.3 Lecture Checklist . . . 19

Learning Objectives

By the end of this lecture, you should be able to:

1. Explain why array element access (`arr[i]`) is $O(1)$ using the address formula.
2. Distinguish between static and dynamic arrays and explain when to use each.
3. Write and interpret Big-O notation for algorithm complexity.
4. Perform exact primitive-operation counting for simple loops and derive the closed-form $T(n)$.
5. Apply asymptotic simplification rules to reduce $T(n)$ to its Big-O class.
6. Explain the doubling strategy and why `pushBack` has amortized $O(1)$ cost.
7. Compare the $O(n^3)$ naive and $O(n^2)$ sort-plus-two-pointer solutions to the 3-Sum problem.
8. Set up recurrence relations for divide-and-conquer algorithms and solve them using the recursion-tree method.
9. Implement a resizable `DynArray` struct with `pushBack`, `insertAt`, and `removeAt` using raw C++ arrays.

Opening Story: The Shelf and the Catalog

Story

Imagine two librarians. The first stores books on a long, unbroken shelf — every book is assigned a sequential slot number. She can hand you any book in seconds because she simply counts slots. But if you ask her to insert a new book in the middle, she has to shift every book to the right of that slot one position over.

The second librarian uses a card catalog — each card points to a box somewhere in a sprawling warehouse. She can insert a new card in the middle of the catalog easily, but retrieving a book means walking to its box, wherever it is.

These two librarians describe the two fundamental storage strategies in computing: the *array* (contiguous, indexed, fast random access, expensive insertion) and the *linked list* (scattered, slow traversal, cheap insertion at known position). Today we master the first librarian and learn to measure *how fast* either strategy really is.

1 Arrays Revisited: Contiguous Memory

Key Concept

When the compiler sees `int arr[5];`, it reserves five consecutive `int`-sized blocks in memory. The hardware can compute the address of any element in a single arithmetic step:

$$\text{address}(\texttt{arr}[i]) = \text{base address} + i \times \texttt{sizeof(int)}$$

This formula is $O(1)$—constant time—regardless of how large the array is. That is the superpower of the array.

Memory layout of `int arr[6] = {10, 20, 30, 40, 50, 60};`

```
Index:    0      1      2      3      4      5
Address: 0x100  0x104  0x108  0x10C  0x110  0x114
Value:  [ 10 ] [ 20 ] [ 30 ] [ 40 ] [ 50 ] [ 60 ]
           ^
           |
          arr (pointer to first element)

arr[i]  ==  *(arr + i)
Address of arr[i]  =  base + i * sizeof(int)
                   =  0x100 + i * 4
```

Key Concept

Random access is $O(1)$ because the address is *computed*, not searched.

1.1 Static Arrays

Definition 1.1 (Static Array). *A **static array** is a fixed-size, contiguous block of memory whose size is determined at* compile time. *All elements occupy adjacent memory locations; the name of the array decays to a pointer to its first element.*

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[6] = {10, 20, 30, 40, 50, 60};
    int n      = 6;

    // O(1) access
    cout << "arr[3] = " << arr[3] << "\n";    // 40

    // O(n) traversal
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
```

```
    cout << "\n";

    // O(n) linear search
    int target = 30;
    int found  = -1;
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) { found = i; break; }
    }
    cout << "Found " << target << " at index " << found << "\n";

    return 0;
}
```

Listing 1: Static array — access, traversal, and search

Warning

Static arrays have three fundamental limitations:

- The size must be known at **compile time**.
- The array **cannot grow or shrink** at runtime.
- Declaring a "large enough" array wastes memory and may still be too small.

These limitations motivate **dynamic arrays**.

1.2 Dynamic Arrays

Definition 1.2 (Dynamic Array)**.** *A **dynamic array** is a contiguous block of memory allocated on the* heap *at runtime with* `new[]`*. The size is not fixed at compile time and can be replaced with a larger allocation when growth is needed.*

```
// STATIC -- size known at compile time
int fixed[100];                // Stack, 100 ints, fixed size

// DYNAMIC -- size known at runtime
int n;
cin >> n;
int* dyn = new int[n];         // Heap, n ints, determined at
    runtime
// ... use dyn ...
delete[] dyn;                  // Must free heap memory!

// PROBLEM: what if we need more than n elements?
// Solution: allocate a bigger array, copy data, free old one
int* bigger = new int[n * 2];
copy(dyn, dyn + n, bigger);  // std::copy from <algorithm>
delete[] dyn;
dyn = bigger;
n   = n * 2;
```

Listing 2: Dynamic array allocation, comparison with static, and resize

Property	Static	Dynamic
Memory location	Stack	Heap
Size determined	Compile time	Runtime
Manual free?	No	Yes — `delete[]`
Speed	Faster	Slightly slower
Flexibility	None	Can resize

Warning

Every `new[]` **must** be paired with exactly one `delete[]`. Failing to do so causes a **memory leak**.

```
#include <iostream>
#include <algorithm>     // std::copy
using namespace std;

int main() {
    int capacity = 4;
    int size     = 0;
    int* data    = new int[capacity];

    // Fill to capacity
    for (int i = 0; i < 4; i++) {
        data[size++] = (i + 1) * 10;
    }
    // data = [10, 20, 30, 40], size=4, capacity=4

    // Need to add one more -- resize manually
    int* newData = new int[capacity * 2];     // double capacity
    copy(data, data + size, newData);         // copy old data
    delete[] data;                            // free old array
    data     = newData;
    capacity = capacity * 2;

    data[size++] = 50;    // now there is room

    for (int i = 0; i < size; i++)
        cout << data[i] << " ";
    cout << "\n";         // 10 20 30 40 50

    delete[] data;
    return 0;
}
```

Listing 3: Manual resize: allocate, copy, free old buffer

2 Algorithm Analysis: Big-O Notation

Story

Speed matters, but raw time in milliseconds depends on the computer, the operating system, and even thermal throttling. What we really want is a measurement that *scales*: "as the input doubles, how does the work grow?" Big-O notation gives us a language for answering exactly that question — independent of hardware.

2.1 Definition

Definition 2.1 (Big-O Notation)**.** *We say $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that*

$$f(n) \leq c \cdot g(n) \quad \textit{for all } n \geq n_0.$$

Intuitively: $g(n)$ is an upper bound *on the growth rate of $f(n)$, ignoring constant factors and small inputs.*

2.2 Common Complexity Classes

Notation	Name	Example	Ops at $n{=}10^6$
$O(1)$	Constant	Array index access	1
$O(\log n)$	Logarithmic	Binary search	20
$O(n)$	Linear	Linear search / scan	10^6
$O(n \log n)$	Linearithmic	Merge sort, Heap sort	$\approx 2 \times 10^7$
$O(n^2)$	Quadratic	Selection/Insertion sort	10^{12}
$O(n^3)$	Cubic	Naive matrix multiply	10^{18}
$O(2^n)$	Exponential	Subset enumeration	∞ (unusable)
$O(n!)$	Factorial	Brute-force permutation	∞ (unusable)

Key Concept

Rule of thumb for this course: $O(n \log n)$ is usually "good enough". $O(n^2)$ is acceptable for $n \leq 10{,}000$.

2.3 Array Operation Complexities

Operation	Best Case	Worst Case	Why?
Access `arr[i]`	$O(1)$	$O(1)$	Address is computed
Search (unsorted)	$O(1)$	$O(n)$	Must scan all n
Search (sorted, binary)	$O(1)$	$O(\log n)$	Halve search space each step
Insert at end	$O(1)$	$O(1)^*$	*If space available
Insert at position i	$O(1)$	$O(n)$	Shift $n{-}i$ elements right
Delete at position i	$O(1)$	$O(n)$	Shift $n{-}i$ elements left
Resize (copy to new array)	$O(n)$	$O(n)$	Must copy all elements

Key Concept

Key insight: Access is fast ($O(1)$) because contiguous memory allows *arithmetic address calculation*. Insertion and deletion in the middle are expensive ($O(n)$) because contiguous memory must *stay* contiguous.

2.4 Quick Analysis Practice

Practice Exercise

Determine the Big-O complexity of each snippet below (answers at the bottom of this box).

```
// Snippet A
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
```

```
// Snippet B
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (arr[i] == arr[j] && i != j) count++;
    }
}
```

```
// Snippet C
int i = 0, j = n - 1;
while (i < j) {
    swap(arr[i++], arr[j--]);
}
```

Answers: A is $O(n)$ — single loop, n iterations. B is $O(n^2)$ — nested loops, $n \times n$ iterations. C is $O(n/2) = O(n)$ — the two pointers meet in the middle.

3 Exact Operation Counting

Key Concept

Cost model: Every primitive operation costs 1 unit.

Statement type	Example	Cost
Variable initialisation	`int i = 0`	1 assign
Arithmetic operation	`i + 1`, `n - 2`	1 per op
Comparison	`i < n`, `a == b`	1 per check
Array access	`arr[i]`	1 per access
Increment / decrement	`i++`, `j-`	1 per op
Assignment	`x = v`	1 per assign

Loop overhead is NOT free. For `for (init; cond; update) { body }` that executes m body iterations:

- `init` (e.g. `int i = 0`): **1 op** — executed exactly once.
- `cond` (e.g. `i < n`): $m + 1$ **ops** — m true checks + 1 final false check.
- `update` (e.g. `i++`): m **ops** — once per body execution.
- `body`: $m \times c_{\text{body}}$ ops.

$$\text{Total for one } \texttt{for} \text{ loop} = 2m + 2 + m \cdot c_{\text{body}}$$

Key Concept

Asymptotic simplification rules:

- Drop lower-order terms: $n^2 + 5n + 7 \;\rightarrow\; O(n^2)$
- Drop constant factors: $\frac{3}{2}n^3 - 2n^2 \;\rightarrow\; O(n^3)$
- Sequential blocks *add*: $O(n) + O(n^2) = O(n^2)$
- Nested loops *multiply*: outer $O(n) \times$ inner $O(n) = O(n^2)$
- Loop with halving: "`i = i / 2` each iteration" $\rightarrow O(\log n)$

Methodology: how to derive $T(n)$ rigorously

1. Annotate every line with its per-execution cost.
2. For each loop, determine its exact iteration count.
3. Sum all costs: loop init once; cond $\times(m{+}1)$; update $\times m$; body $\times m$.
4. Evaluate nested sums using standard formulas: $\displaystyle\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$
5. Collect into $T(n)$, then apply asymptotic simplification.

3.1 Worked Example: Double-Nested Loop

Example

[Complete operation count for a double-nested loop]

```
int count = 0;                                  // (a)
for (int i = 0; i < n; i++) {                  // (b)
    for (int j = i + 1; j < n; j++) {          // (c)
        if (arr[i] > arr[j]) count++;           // (d)
    }
}
```

Let $A = n$ (outer iterations) and $B = \frac{n(n-1)}{2}$ (total inner iterations).

Line	Operation	Executions	Cost/exec
(a)	`count = 0`	1	1
(b)	outer init: `i = 0`	1	1
(b)	outer cond: `i < n`	$A + 1$	1
(b)	outer incr: `i++`	A	1
(c)	inner init: `j = i+1` (1 add + 1 assign)	A	2
(c)	inner cond: `j < n`	$B + A$	1
(c)	inner incr: `j++`	B	1
(d)	`arr[i]`, `arr[j]`, `>` (2 reads + 1 cmp)	B	3
(d)	`count++` (worst case)	B	1

Summing all costs:

$$T(n) = 1 + 1 + (n + 1) + n + 2n + (B + n) + B + 3B + B = 3 + 5n + 6B$$

Substituting $B = \frac{n(n-1)}{2}$:

$$T(n) = 3 + 5n + 3n(n - 1) = 3n^2 + 2n + 3$$

Verification ($n = 4$): $3(16) + 2(4) + 3 = 59$ ops ✓
Asymptotic simplification: $T(n) = 3n^2 + 2n + 3 = \boxed{O(n^2)}$

4 Amortized Analysis: The Doubling Strategy

Story

Imagine you are filling a jar with marbles. The jar holds 4 marbles. When it is full, you do not buy a slightly bigger jar — you buy a jar *twice the size*, pour everything in, and keep going. Buying new jars is expensive, but you do it so rarely that the *average* cost per marble is still tiny.
This is exactly how `std::vector` (and our `DynArray`) works. Each `pushBack` that triggers a resize is expensive ($O(n)$), but resizes happen so rarely that the *amortized* (average) cost per `pushBack` is $O(1)$.

Definition 4.1 (Amortized $O(1)$)**.** *An operation has* ***amortized*** $O(1)$ *cost if a sequence of n such operations takes $O(n)$ total time, even if individual operations can be expensive. The cost of expensive operations is "spread out" over the cheaper ones.*

Doubling strategy trace:

Operation	State	Array	Cost
`pushBack(10)`	size=1, cap=2	`[10, _]`	1
`pushBack(20)`	size=2, cap=2	`[10, 20]`	1
`pushBack(30)`	size=3, cap=4	`[10, 20, 30, _]`	$1 + 2$ (resize)
`pushBack(40)`	size=4, cap=4	`[10, 20, 30, 40]`	1
`pushBack(50)`	size=5, cap=8	`[10, 20, 30, 40, 50, _ _ _]`	$1 + 4$ (resize)
`pushBack(60)`	size=6, cap=8	...	1
`pushBack(70)`	size=7, cap=8	...	1
`pushBack(80)`	size=8, cap=8	...	1
`pushBack(90)`	size=9, cap=16	...	$1 + 8$ (resize)

Key Concept

Total copy work after n pushbacks:

$$1 + 2 + 4 + 8 + \cdots + \frac{n}{2} = n - 1 < n$$

Total cost for n pushbacks $< 2n$. Dividing by n: $\frac{2n}{n} = 2 = O(1)$ **amortized per pushBack**.

Warning

Do not confuse "amortized $O(1)$" with "always $O(1)$". Any *single* `pushBack` that triggers a resize is genuinely $O(n)$, but that is rare. The *average* cost over many operations is $O(1)$.

5 Worked Example: The 3-Sum Problem

Key Concept

Problem: Given an integer array, count all triples (i, j, k) with $i < j < k$ such that $\texttt{arr}[i] + \texttt{arr}[j] + \texttt{arr}[k] = 0$.

5.1 Naive Approach — $O(n^3)$

```cpp
#include <iostream>
using namespace std;

int threeSum(int* arr, int n) {
    int count = 0;
    for (int i = 0; i < n - 2; i++) {            // outer
        for (int j = i + 1; j < n - 1; j++) {    // middle
            for (int k = j + 1; k < n; k++) {    // inner
                if (arr[i] + arr[j] + arr[k] == 0)
                    count++;
            }
        }
    }
    return count;
}

int main() {
    int a[] = {-4, -1, -1, 0, 1, 2};
    int n   = 6;
    cout << "Zero-sum triplets: " << threeSum(a, n) << "\n";   // 2
    return 0;
}
```

Listing 4: Naive 3-Sum: try every triple — $O(n^3)$

Number of triples examined:

$$C = \binom{n}{3} = \frac{n(n-1)(n-2)}{6} \quad \Rightarrow \quad T(n) = \frac{3}{2}n^3 - \frac{3}{2}n^2 + n - 4 = O(n^3)$$

Wall-clock impact at 10^9 ops/second:

n	Approx. time
100	< 0.002 s
1,000	≈ 1.5 s
10,000	$\approx 1{,}500$ s — completely unusable!

5.2 Optimised Approach — $O(n^2)$: Sort + Two Pointers

Key Concept

Key idea: Sort the array first ($O(n \log n)$), then for each fixed index i, use two pointers `lo` and `hi` that scan inward. At each step, if the sum is too small, advance `lo`; if too large, retreat `hi`. Each pointer moves at most n total steps per outer iteration, giving $O(n)$ per i.

```cpp
#include <iostream>
#include <algorithm>    // std::sort
using namespace std;

int threeSumFast(int* arr, int n) {
    sort(arr, arr + n);         // O(n log n)
    int count = 0;
    for (int i = 0; i < n - 2; i++) {            // O(n)
        int lo = i + 1, hi = n - 1;
        while (lo < hi) {                         // O(n) total per i
            int s = arr[i] + arr[lo] + arr[hi];
            if      (s == 0) { count++; lo++; hi--; }
            else if (s  < 0) { lo++; }
            else             { hi--; }
        }
    }
    return count;   // O(n^2) overall
}

int main() {
    int a[] = {-4, -1, -1, 0, 1, 2};
    int n   = 6;
    cout << "Zero-sum triplets (fast): " << threeSumFast(a, n) << "\n";
    return 0;
}
```

Listing 5: Optimised 3-Sum with sort + two pointers — $O(n^2)$

Total complexity:

$$T_{\text{fast}}(n) = O(n \log n) + O(n^2) = O(n^2)$$

Approach	Leading term	At $n = 10{,}000$
Naive $O(n^3)$	$\frac{3}{2}n^3$	1.5×10^{12} ops ($\approx 1{,}500$ s)
Fast $O(n^2)$	$\frac{11}{2}n^2$	5.5×10^{8} ops (< 1 s)
Speedup		$\approx$ **2,700**$\times$

Key Concept

Key lesson: Sorting upfront ($O(n \log n)$) is often worth paying for, because it enables a linear inner pass instead of a quadratic or cubic one.

Practice Exercise

Trace `threeSumFast` on the array `[-2, 0, 1, 1, 2]` (sorted). For $i = 0$ (`arr[0]=-2`), show how `lo` and `hi` move and what triples are found. How many total triplets sum to zero?

6 Recurrence Relations and Merge Sort

Story

A naive nested-loop approach to sorting takes $O(n^2)$ steps — painful for millions of elements. Merge sort achieves $O(n \log n)$ through *divide and conquer*: split the array in half, sort each half recursively, then merge. Analyzing it requires a new tool: **recurrence relations**. The full implementation will be studied after midterm; today we focus on deriving its complexity.

6.1 Merge Sort Recurrence

Algorithm sketch (for analysis):

```
mergeSort(arr, lo, hi):
    if hi - lo <= 0: return                // base case: 1 element, O(1)
    mid = (lo + hi) / 2
    mergeSort(arr, lo, mid)                // T(n/2): sort left half
    mergeSort(arr, mid+1, hi)              // T(n/2): sort right half
    merge(arr, lo, mid, hi)               // O(n):   scan & merge both halves
```

Definition 6.1 (Recurrence Relation for Merge Sort)**.**

$$T(n) = 2\,T\left(\frac{n}{2}\right) + cn, \qquad T(1) = c_0$$

where c is the per-element cost of the merge step and c_0 is a constant for the base case.

6.2 Solving the Recurrence: Recursion Tree Method

Expand level by level (work done *outside* the recursive calls at each level):

Level	Subproblems	Work at this level
0	1 problem of size n	cn
1	2 problems of size $n/2$	$2 \cdot c(n/2) = cn$
2	4 problems of size $n/4$	$4 \cdot c(n/4) = cn$
$\vdots$	$\vdots$	$\vdots$
k	2^k problems of size $n/2^k$	$2^k \cdot c(n/2^k) = cn$

The recursion bottoms out when $n/2^k = 1$, i.e., $k = \log_2 n$.
Total cost:

$$T(n) = \underbrace{cn + cn + \cdots + cn}_{\log_2 n \text{ levels}} + n \cdot c_0 = cn \log_2 n + c_0 n = O(n \log n)$$

Key Concept

Concrete impact at $n = 1{,}000{,}000$:

- A naive $O(n^2)$ algorithm: $n^2/2 \approx 5 \times 10^{11}$ ops
- Merge sort $O(n \log n)$: $n \log_2 n \approx 2 \times 10^7$ ops
- $\approx 25{,}000\times$ **fewer operations**

6.3 Recurrence Patterns Summary

Algorithm	Recurrence	Complexity
Binary search	$T(n) = T(n/2) + O(1)$	$O(\log n)$
Merge sort	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$
Naive Fibonacci	$T(n) = 2T(n-1) + O(1)$	$O(2^n)$

Key Concept

The pattern: *number of subproblems* $\times$ *size fraction* $+$ *work per level* determines the final complexity.

Practice Exercise

For each recurrence, determine its Big-O class using the recursion-tree method:

1. $T(n) = T(n/3) + O(1)$
2. $T(n) = 2T(n/3) + O(n)$
3. $T(n) = 4T(n/2) + O(n)$

7 Implementing a Resizable Array (Procedural Style)

Story

`std::vector` is C++'s battle-tested resizable array. Before using it, we build our own scaled-down version — *without classes or templates*. All state lives in a plain `struct`; all behaviour lives in free functions operating on raw pointers. This is the style of C, embedded systems, and any context where OOP overhead is undesirable.

7.1 Interface Design

Key Concept

Data structure:

```
struct DynArray {
    int* data;       // pointer to heap-allocated array
    int  size;       // number of elements currently stored
    int  capacity;   // total allocated slots

    // Invariant: size <= capacity
    // All heap memory owned by this struct; caller must call
       freeArray.
};
```

Interface (free functions):

Function	Complexity
`initArray(a)`	$O(1)$
`freeArray(a)`	$O(1)$
`growArray(a)`	$O(n)$ — internal, called by `pushBack`
`pushBack(a, value)`	Amortized $O(1)$
`insertAt(a, index, v)`	$O(n)$
`removeAt(a, index)`	$O(n)$
`getAt(a, index)`	$O(1)$
`setAt(a, index, v)`	$O(1)$
`printDynArr(a)`	$O(n)$

Key Concept

Why keep `size` and `capacity` separate?

- `size` = number of elements *currently in use*.
- `capacity` = number of slots *already allocated*.
- When `size == capacity`, the next `pushBack` must call `growArray` to allocate more memory.

7.2 Full Implementation

```
#include <iostream>
using namespace std;

// ---- Data structure ----------------------------------------------------

struct DynArray {
    int* data;
```

```
    int  size;
    int  capacity;
};

// ---- Initialise and destroy ----------------------------------------

void initArray(DynArray& a) {
    a.data     = nullptr;
    a.size     = 0;
    a.capacity = 0;
}

void freeArray(DynArray& a) {
    delete[] a.data;
    a.data     = nullptr;
    a.size     = 0;
    a.capacity = 0;
}

// ---- Internal: grow (double capacity, copy, free old) -----------------
// Called only when size == capacity -- O(n)
void growArray(DynArray& a) {
    int  newCap  = (a.capacity == 0) ? 1 : a.capacity * 2;
    int* newData = new int[newCap];
    for (int i = 0; i < a.size; i++)    // copy existing elements
        newData[i] = a.data[i];
    delete[] a.data;                    // release old buffer
    a.data     = newData;
    a.capacity = newCap;
}

// ---- Modifiers ------------------------------------------------------

// Append at end -- amortised O(1)
void pushBack(DynArray& a, int value) {
    if (a.size == a.capacity) growArray(a);  // O(n) but rare
    a.data[a.size++] = value;
}

// Insert value at index -- O(n): shift elements right
void insertAt(DynArray& a, int index, int value) {
    if (index < 0 || index > a.size) return;   // bounds check
    if (a.size == a.capacity) growArray(a);
    // Shift right-to-left to avoid overwriting uncopied data
    for (int i = a.size; i > index; i--)
        a.data[i] = a.data[i - 1];
    a.data[index] = value;
    a.size++;
```

```
}

// Remove element at index -- O(n): shift elements left
void removeAt(DynArray& a, int index) {
    if (index < 0 || index >= a.size) return;  // bounds check
    for (int i = index; i < a.size - 1; i++)
        a.data[i] = a.data[i + 1];
    a.size--;
}

// ---- Accessors ----------------------------------------------------------

int getAt(const DynArray& a, int index) {
    return a.data[index];    // caller responsible for valid index
}

void setAt(DynArray& a, int index, int value) {
    a.data[index] = value;   // caller responsible for valid index
}

// ---- Print ---------------------------------------------------------------

void printDynArr(const DynArray& a) {
    cout << "[";
    for (int i = 0; i < a.size; i++) {
        if (i > 0) cout << ", ";
        cout << a.data[i];
    }
    cout << "]  (size=" << a.size
         << ", cap=" << a.capacity << ")\n";
}
```

Listing 6: `DynArray` — complete procedural implementation

```
int main() {
    DynArray v;
    initArray(v);

    // pushBack -- observe growing capacity
    for (int i = 1; i <= 9; i++) {
        pushBack(v, i * 10);
        printDynArr(v);            // watch size and cap grow
    }

    cout << "\n--- insertAt(2, 5) ---\n";
    insertAt(v, 2, 5);
    printDynArr(v);

    cout << "\n--- removeAt(0) ---\n";
    removeAt(v, 0);
```

```
    printDynArr(v);

    cout << "\n--- element access ---\n";
    cout << "getAt(1) = " << getAt(v, 1) << "\n";
    setAt(v, 1, 999);
    cout << "after setAt(1, 999): ";
    printDynArr(v);

    freeArray(v);   // release heap memory
    return 0;
}
```

Listing 7: Testing `DynArray`

7.3 Critical Bug: Insertion Direction

Warning

Wrong shift direction (left-to-right) in `insertAt`:

```
// WRONG: overwrites before copying!
// Inserting 99 at index 1 in [10, 20, 30, 40]
for (int i = index; i < a.size; i++) {
    a.data[i + 1] = a.data[i];   // step 1: data[2] = data[1]
        = 20  OK
                                  // step 2: data[3] = data[2]
                                      = 20  !!
                                  //          data[2] was
                                     already overwritten!
}
```

Correct shift direction (right-to-left):

```
// CORRECT: shift from the end inward
// [10, 20, 30, 40] --> insert 99 at index 1
for (int i = a.size; i > index; i--) {
    a.data[i] = a.data[i - 1];   // step 1: data[4] = data[3]
        = 40
                                  // step 2: data[3] = data[2]
                                      = 30
                                  // step 3: data[2] = data[1]
                                      = 20
}
a.data[index] = 99;               // Result: [10, 99, 20, 30,
   40]
```

Rule: When inserting, always shift **right-to-left** (from the end toward the insertion point).

Practice Exercise

Trace `insertAt(v, 1, 99)` manually on the array `[10, 20, 30, 40]` (size=4, capacity=4).

1. Does `growArray` get called? Why or why not?
2. Show the array state after each shift step (right-to-left).
3. What is the final array?

8 Summary

8.1 Complexity Summary Table

Operation	Best	Average	Worst
Access `arr[i]`	$O(1)$	$O(1)$	$O(1)$
`pushBack` (amortized)	$O(1)$	$O(1)$	$O(1)^*$
`insertAt` position i	$O(1)$	$O(n)$	$O(n)$
`removeAt` position i	$O(1)$	$O(n)$	$O(n)$
`growArray` (resize)	$O(n)$	$O(n)$	$O(n)$
3-Sum naive	—	$O(n^3)$	$O(n^3)$
3-Sum sort + two-pointer	—	$O(n^2)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Table 1: Array and algorithm complexities covered in Lecture 03. (*Individual resize is $O(n)$; amortized over all pushbacks is $O(1)$.)

8.2 Common Pitfalls Checklist

8.3 Lecture Checklist

- ✓ Contiguous memory layout — why `arr[i]` is $O(1)$
- ✓ Static vs dynamic arrays — heap allocation with `new[]`
- ✓ Big-O notation and common complexity classes
- ✓ Step-by-step exact operation counting and asymptotic simplification
- ✓ Amortized $O(1)$: the doubling strategy
- ✓ 3-Sum: $O(n^3)$ naive vs $O(n^2)$ sort + two-pointer
- ✓ Recurrence relations: $T(n) = 2T(n/2) + cn \Rightarrow O(n \log n)$
- ✓ `DynArray` struct + free functions: `pushBack`, `insertAt`, `removeAt`

Pitfall	**Prevention**
Out-of-bounds access	Bounds-check `index` in every accessor and modifier
Wrong shift direction in `insertAt`	Shift **right-to-left** (from `size` down to `index`)
Integer overflow at midpoint	Use `lo + (hi - lo) / 2`, not `(lo + hi) / 2`
Forgetting `a.size++` in `pushBack`	Test: n pushbacks must give `a.size == n`
Memory leak in `growArray`	Call `delete[] a.data` *after* copying, then reassign
Forgetting `freeArray` at end of scope	Every `initArray` must be paired with `freeArray`
Using `a.data` after `freeArray`	Set `a.data = nullptr` inside `freeArray`
Solving the wrong recurrence	Identify: # subproblems, size fraction, and work per level

Table 2: Common bugs in array implementation and algorithm analysis

What's Next?

Story

Arrays are fast and simple, but their Achilles' heel is insertion and deletion in the middle: $O(n)$ every time, because of that unbreakable contiguity requirement. What if we could build a data structure where inserting at any position is $O(1)$ — at the cost of losing the ability to jump directly to index i?

That trade-off is exactly the *linked list* — the subject of Lecture 04. The shelf becomes a chain of scattered index cards, each pointing to the next. You lose the address formula, but gain the freedom to insert between any two cards by changing a single pointer.

See you next time. Bring your pencil for pointer diagrams.

"An algorithm must be seen to be believed."
— Donald E. Knuth